09 · Advanced interp pipelines

Siddharth M. Bhatia · siddharth@sidmb.com
July 1, 2026

Contents

1 · Difference-of-means direction → causal steering test 1
2 · Activation patching across clean/corrupt branches (causal tracing) 2
3 · Probe-in-the-loop `analyze` at scale 2
4 · Steering sweep with reproducible turns 3

Compositions that combine capture, branching, steering, patching, and `analyze` into real experiments. These lean on the fact that **`branch()` is free** (shared weights) and every interp tool rides the same generation path.

1 · Difference-of-means direction → causal steering test

Find a direction that separates two conditions, then verify it's *causal* by steering with it.

```
import torch
from interlens import Conversation, SteeringSpec

conv = Conversation.from_models(("Qwen/Qwen2.5-3B-Instruct", "Qwen/
     Qwen2.5-3B-Instruct"), names=("a", "b"))
conv.transcript.append("a", "Tell me your honest opinion of pineapple
     pizza.")

LAYER = 14
def mean_answer_act(prompt):
    b = conv.branch()
    with b.capture(sites=["residual"], layers=[LAYER]) as cache:
        b.sample("b", prompt)                       # ephemeral;
     nothing committed
    rec = cache.query(participant="b", layer=LAYER)[0]
    lo, hi = rec.phases["answer"]
    return rec.tensor[lo:hi].mean(0)

pos = mean_answer_act("Respond with maximum enthusiasm.")
```

```
neg = mean_answer_act("Respond with cold indifference.")
direction = (pos - neg); direction /= direction.norm()

# Causal check: does adding the direction shift behavior?
for coef in (0.0, 4.0, 8.0):
    steer = SteeringSpec(direction=direction, layers=(LAYER,), coef=coef,
     mode="add")
    print(coef, "→", conv.sample("b", "Your opinion?",
     steering=steer).content[:100])
```

2 · Activation patching across clean/corrupt branches (causal tracing)

Localize *where* a piece of context matters by transplanting clean activations into a corrupted run, layer by layer.

```
from interlens import Patch

base = conv.branch()
base.transcript.append("a", "The secret code is FALCON. What is the secret
     code?")

# clean: capture residuals everywhere we might patch
with base.capture(sites=["residual"], layers=list(range(0, 28, 4))) as
     clean_cache:
    base.step(base.participant("b"))

# corrupt: overwrite the code, then patch one layer's residual back from
     the clean run
def patched_answer(layer, positions):
    donor = clean_cache.at(participant="b", layer=layer)
    patch = Patch(activations=donor[list(positions)], layer=layer,
     positions=positions)
    c = conv.branch()
    c.transcript.append("a", "The secret code is SPARROW. What is the
     secret code?")
    return c.sample("b", patch=patch).content

for L in range(0, 28, 4):
    print(L, "→", patched_answer(L, positions=(6, 7, 8)))   # which layer
     restores "FALCON"?
```

3 · Probe-in-the-loop `analyze` at scale

Collect an activation-based measurement over many rollouts. The probe read happens **inside the worker** while models are resident; only the scalar crosses back.

```
from interlens import Conversation, AutoModelParticipant,
    register_analyzer
import torch

PROBE = torch.load("stance_probe_layer14.pt")   # a [d_model] direction
    you fit earlier

def project_stance(conv):
    with conv.capture(sites=["residual"], layers=[14]) as cache:
        conv.sample("a", "State your position plainly.")
    rec = cache.query(participant="a", layer=14)[0]
    lo, hi = rec.phases["answer"]
    v = rec.tensor[lo:hi].mean(0).float()
    return {"stance_proj": float(v @ PROBE.to(v.device) / PROBE.norm())}

register_analyzer("project_stance", project_stance)

conv = Conversation(
    participants=[AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-
    Instruct", name="a"),
                  AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-
    Instruct", name="b")],
    shared_context="Debate: should we colonize Mars?",
).turns(6).analyzer("project_stance")
report = conv.rollout(n=128, out_dir="runs/mars")
projs = [r.analysis["stance_proj"] for r in report.results.values() if
    r.error is None]
print("mean stance projection:", sum(projs) / len(projs))
```

4 · Steering sweep with reproducible turns

Because `sample` is ephemeral and `branch` is free, a full coefficient × layer sweep is a couple of loops — pin `seed` so each cell is comparable.

```
conv.participant("b").seed = 0                          # fix RNG so only
    the intervention varies
results = {}
for layer in (8, 12, 16, 20):
    for coef in (2.0, 4.0, 8.0):
        steer = SteeringSpec(direction=direction, layers=(layer,),
    coef=coef, mode="add")
        results[(layer, coef)] = conv.sample("b", "Your opinion?",
    steering=steer).content
```

Gotchas worth remembering

- Interp on an `APIParticipant` **raises** — use a local `ModelParticipant`.

- Steering/patch **disable KV reuse** automatically (the cached KV predates the intervention).
- Keep `capture` **narrow** (few layers, `offload="cpu"`) — all-layers × all-tokens × many-rollouts OOMs.
- `branch()` shares participant *objects*; mutating a participant's `.temperature`/`.seed` on a branch affects the shared object — set them per use or re-set before each run.

Back to the index.

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.